Python Basics

Philipp Gloor

Bytes & Beyond

© 2026 Bytes & Beyond GmbH. All rights reserved.

Table of Contents

- General Introduction
- Introduction to Programming
- Fundamental Concepts
  - Building Blocks
  - Input and Output
  - Conditionals: if/else/elif
  - Naming Conventions & Debugging
  - Lists: []
  - Immutables: Tuples () and Strings
  - Iteration: for/while
  - Modules & Imports
  - Reading Data
  - Functions
  - Dictionaries: {}
- Persistence

1

General Introduction

General Introduction - About me

Education

- 2012 – Bachelor of Science in Physics
- 2016 – Master of Science in Computational Science

Work

- 2014 – 2016: Software engineer CERN (remote)
- 2016 – 2021: PDF Tools AG
- 2021 – now: Zurich Instruments

Programming experience

- Python, C++, C#, Java, TypeScript, JavaScript

Email

- philipp.gloor@proton.me

General Introduction - Learning targets

After this course...

- ... you will be able to write a Python program that reads input, makes decisions, and produces output
- ... you will be able to store and process data with lists, dictionaries, and loops
- ... you will be able to break a program into functions and reuse them instead of repeating code
- ... you will be able to save a program's data to a file and load it again the next time it runs

But you will only have started to scratch the surface.

2

Introduction to Programming

A program is a sequence of instructions that tells the computer exactly what to do.

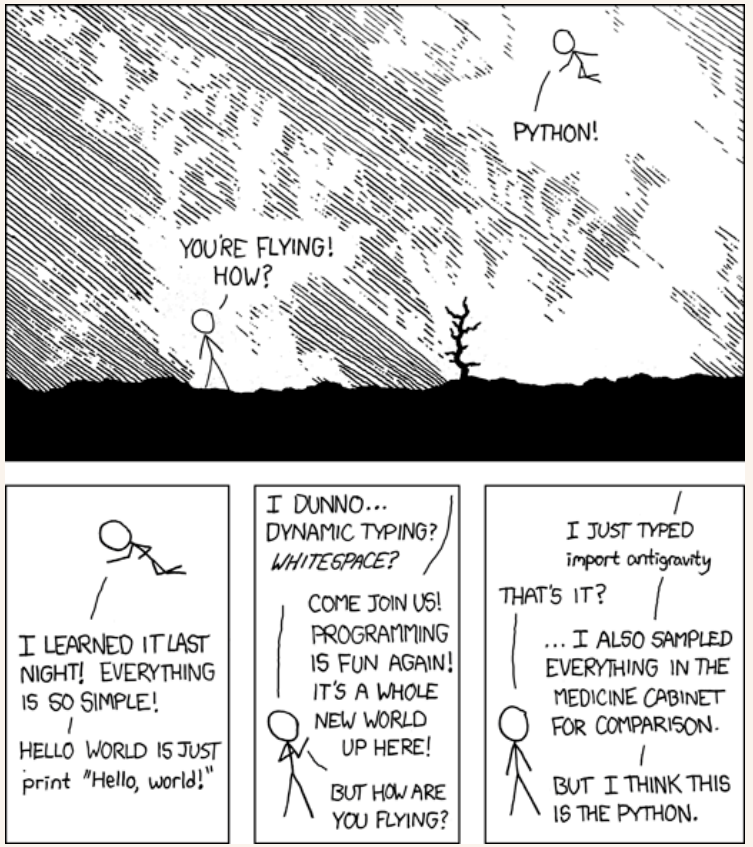
Most programs follow the same shape

- **Input**: data from the keyboard, a file, the internet, ...
- **Processing**: the instructions that transform the input - also called an **algorithm**
- **Output**: the result - printed to the screen, saved to a file, ...

You rarely write the processing from scratch: **libraries** give you processing other people already wrote and tested. But you will have to write the code that ties it all together.

Introduction to Programming - Why Python?

- High-level programming language
- “Simple” syntax
- Cross-platform - A script written on a Windows computer also runs on Linux & Mac
- Interpreted - easy to run
- Many libraries available



Source: https://xkcd.com/353/

High level languages but not trivial to learn:

Java

```java
public class HelloWorld {
  public static void main(String args[]) {
    System.out.println("Hello World");
  }
}
```

C++

```cpp
#include <iostream>
int main() {
  std::cout << "Hello World\n";
  return 0;
}
```

Or even worse:

Assembly Language

Example of a low level language

```
.LC0:
  .string "Hello world!"
main:
  push rbp
  mov rbp, rsp
  mov edi, OFFSET FLAT:.LC0
  mov eax, 0
  call printf
  mov eax, 0
  pop rbp
  ret
```

And in Python?

```
print("Hello World")
```

Options to run Python code:

- Directly in the Python prompt (REPL - Read, Eval, Print, Loop)
- Write the code into a file and run python with the file
- Use IDE to run Python code

When is the REPL useful?

- Trying out a single line without creating a file for it
- Checking what a piece of code actually returns, right away
- Exploring: what does this function do? what type is this value?

```
>>> 2 + 3
5
>>> len("Hello")
5
```

Introduction to Programming - Development Environment

- Integrated Development Environment (IDE)
- Collection of tools that are commonly used for software development (they make our life easier!)
- Popular IDEs
  - Visual Studio Code - https://code.visualstudio.com
  - JetBrains PyCharm - Community Edition available for free https://jetbrains.com/pycharm/download
- It takes time to get proficient using an IDE

3

Fundamental Concepts

Building Blocks

Building Blocks - Variables

A variable stores a value under a name, so you can reuse and refer to it later - think of it as a labeled box: the name is the label, the value is what's inside.

- Values are assigned with an equal sign. So you can write: `x = 2`
- Like in math, you can build new values from existing variables: `y = x + 2`
  - But unlike in math, this only runs once: if `x` changes afterwards, `y` keeps its old value
- Use descriptive names: your code will be read by you and by others

Building Blocks - Data Types

Numbers

Integers

- 2
- 1000000
- -2

Floats

- 3.2
- 1.333333

Building Blocks - Data Types

Strings

- Text, written with 'single' or "double" quotes - both work the same
  - 'Hello World'
  - "Hello World"
  - "5" - still text, not a number

Booleans

Binary data type: **True** or **False**

Building Blocks - None Type

Sometimes a variable's box exists, but there is nothing in it yet. Python has a special value for exactly that: `None`.

- `None` means “no value” - it is not the same as `0`, an empty string or `False`
- Common example: a person's middle name, when they do not have one
- You assign it on purpose, the same way you would assign any other value

Examples

```
middle_name = None  # We don't know this person's middle name yet
print(middle_name)  # Prints: None

if middle_name == None:
  print("No middle name on file")
```

Building Blocks - Reserved Keywords

You cannot name a variable with these names as they are protected by the language.

```
False      await      else       import     pass
None       break      except     in         raise
True       class      finally    is         return
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
```

Building Blocks - Operators

Order of precedence, highest first (kind of like PEMDAS):

Operator	Common name
`( )`	Parentheses
`**`	Exponentiation
`+ -`	Sign (unary)
`* / % //`	Multiplication, division, modulo, floor division
`+ -`	Addition, subtraction
`< > <= >= != ==`	Comparison
`not`	Logical not
`and`	Logical and
`or`	Logical or



Building Blocks - Operators

Let's go through each of these in detail:

- () - parentheses: Evaluated first, before anything else

```
(2 + 3) * 4  # 20
```

- ** - Exponentiation (power)

```
2 ** 4  # 16
```

Building Blocks - Operators

Order of precedence, continued:

- Unary + -: The sign of a number

```
-7
```

- * / % //: Multiplication, division, modulo, floor division
- / always gives back a float, even if the numbers divide evenly
- // is floor division: divide, then drop everything after the decimal point

```
7 / 2   # 3.5   division
9 // 2  # 4     floor division
```

Building Blocks - Operators

Order of precedence, continued:

- + -: Addition and subtraction

```
5 - 2  # 3
```

- < > <= >= != == Comparisons

```
5 > 3  # True
```

Building Blocks - Operators

Order of precedence, continued:

- **`not`** - Negation: inverts the boolean

```
not True  # False
```

- **`and`** - Both must be true

```
True and False  # False
```

- **`or`** - At least one must be true

```
True or False  # True
```

Combining values

- Strings can be combined (concatenated) with + too
- But Python does not silently convert between numbers and strings

```
salutation = "Hello"
name = "Monty Python"
print(salutation + " " + name)  # "Hello Monty Python"

print(3 + "3")  # TypeError: unsupported operand type(s)
                #            for +: 'int' and 'str'
```

Building Blocks - Augmented Assignment

- A variable can be updated using its own current value
- `+=` is a shorthand for exactly that, and `-=` works the same way

```
counter = 0
counter = counter + 1 # counter is now 1
counter += 1          # shorthand for the same thing, counter is now 2
counter -= 1          # counter is back to 1
```

Building Blocks - Comments

- A comment starts with # and runs to the end of the line
- Comments have no effect on the program - Python ignores them completely
- They can stand on their own line, or follow code on the same line

Good comments explain why, not what

- The code already shows what it does - a comment that just repeats it adds nothing
- Save comments for what the code cannot say on its own: a reason, a warning, a trade-off

```
price = price * 0.9  # bad: multiplies price by 0.9

price = price * 0.9  # good: 10% loyalty discount
```

Building Blocks - Built-in Functions

Functions are self-contained modules of code that accomplish a specific task. Python comes with a whole set of them, ready to use.

Calling a function

- Write its name, then the **arguments** in parentheses
- The parentheses are what actually runs it - `len` on its own does nothing
- Depending on the function there can be no, one, or several arguments

Examples

```
len("Hello")   # 5                 how many characters
type(3.2)      # <class 'float'>   which data type
round(3.7)     # 4                 nearest whole number
max(3, 9, 2)   # 9                 largest of several values
```

Some functions hand a value back

- `len(text)` gives you a number that you can store and use
- The result is lost unless you store it or use it right away

```
text = "Python programming language"

text_length = len(text)  # store the result
print(text_length)       # Prints: 27

print(len(text))         # or use it right away

len(text)                # runs, but the result is thrown away
```

Others just do something

- `print()` writes to the console but hands nothing back (`None`)

Building Blocks - Built-In Functions

https://docs.python.org/3/library/functions.html

Input and Output

Input and Output - Input and Type Conversions

Assume you want to program a calculator. In order to do this the user needs to be able to input their numbers and the program should be able to read this. Here comes the `input` function into play.

```
print("Add two numbers")
a = input("Please enter the first number ")
b = input("Please enter the second number ")

print("The result of the two numbers is: ")
print(a + b) # Something is wrong here
```

Input and Output - Input and Type Conversions

Sometimes it is necessary to convert a variable from one data type to another (if possible). If you read data from a file into python, at first all the data is interpreted as strings even if your file only contains numbers.

In order to do mathematical analysis on these numbers, they need to be converted to the appropriate number type first.

```
int("32")         # 32          converts a string that holds a number to an integer
float("313.333") # 313.333     converts a string that holds a number to a float
str(32)           # "32"        converts a number to a string
int("Hello")      # ValueError: "Hello" doesn't hold a number
```

Input and Output - f-strings

Thanks to the f-string there is at least one less need to use explicit conversion functions:

Examples

```
a = 20
b = 10
res = a + b
print(f"The sum of {a} and {b} is {res}")
# used to look like this:
print("The sum of " + str(a) + " and " + str(b) + " is " + str(res))
```

Rounding for display

- `:.2f` inside the braces shows a number with two decimals
- The number of decimals is up to you: `:.0f`, `:.3f`, …
- This only changes what is **printed** - the value itself does not change

```
average = 4.333333333333333
print(f"Average: {average}")       # Average: 4.333333333333333
print(f"Average: {average:.2f}")   # Average: 4.33
print(average)                     # 4.333333333333333 - still the same value
```

Conditionals: if/else/elif

Conditionals: if/else/elif - Conditionals

- Boolean algebra is a part of mathematics
- Often used in programming
- A boolean expression is either true or false

```
5 == 5 # --> True
5 == 6 # --> False
6 > 4 # --> True
5 >= 8 # --> False
```

if

- The `if` statement defines a condition
- If the condition is true, subsequent statements will be executed
- If the condition is false, subsequent statements will not be executed
- There has to be at least one statement after the condition

```
x = 10
if x > 0:
  print(f"{x} is positive")
if True:
  # This statement will always be executed
  print("Yes")
if False:
  # This statement will never be executed
  print("No")
```

else

- Expression else is executed if the if condition is false
- Can only be used in combination with an `if` statement

```
if x == 0:
  print("x is zero")
else:
  print("x is not zero")
```

Example: Even or Odd

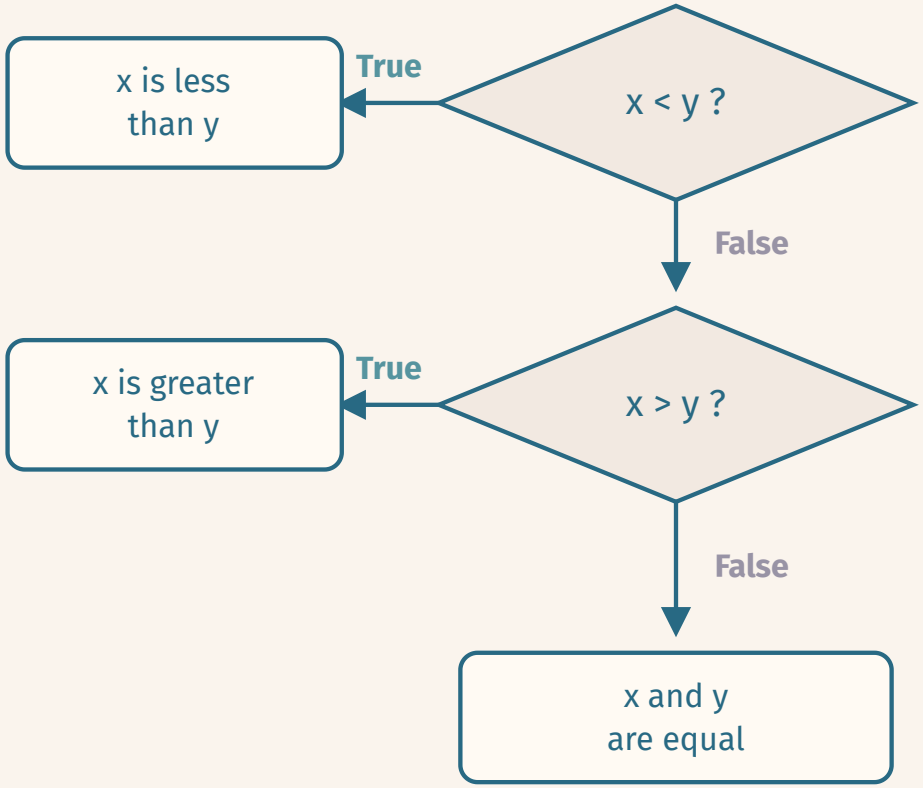
- The % operator gives you the remainder of a division
- A number is even exactly when the remainder of dividing it by 2 is 0

```
if x % 2 == 0:
  print("The number is even")
else:
  print("The number is odd")
```

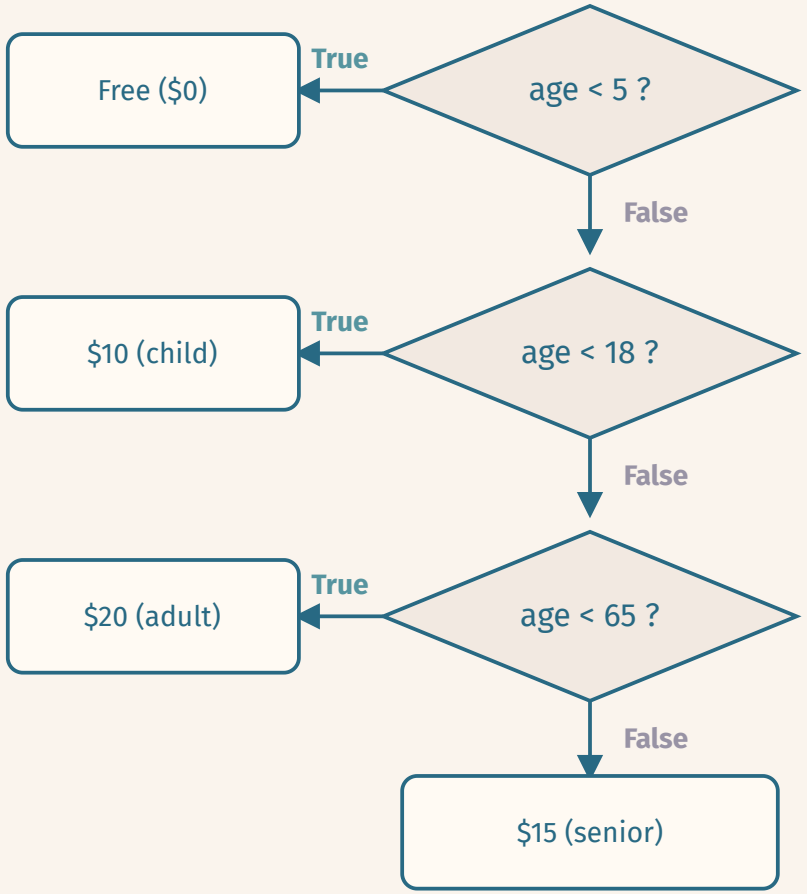
Chained conditionals

- `elif` is used to combine multiple conditions
- The `else` branch is executed when neither `if` nor any of the `elif`s is true.
- Any number of `elif` branches can be used but only one `if` and one `else`

Tracing the `if` / `elif` / `else` chain for `x` and `y`:



Tracing an `if` / `elif` / `elif` / `else` chain: a ticket price based on age




```
age = 70

if age < 5:
    price = 0
elif age < 18:
    price = 10
elif age < 65:
    price = 20
else:
    price = 15

print(price)  # 15
```

Examples

```
if x < y:
  print(f"{x} is less than {y}")
elif x > y:
  print(f"{x} is greater than {y}")
else:
  print(f"{x} and {y} are equal")
```

```
# Python 3
answer = input("Do you like Python?")
if answer == "yes":
  print("That is great!")
else:
  print("That is disappointing!")
```

Nested conditionals

- Conditionals can be nested

```
if x > 0:
  if x < 10:
    print("x is a positive single digit")
```

and

- Deep nesting can be difficult to read
- Use `and` to combine conditionals

```
if x > 0:
  if x < 10:
    print("x is a positive single digit")
# is the same as
if x > 0 and x < 10:
  print("x is a positive single digit")
```

or

- At least one statement must be true for the condition to be true

```
if x > 0 or x < 0:
  print("x is not zero")
```

`not`

- Negation, inverts the boolean.
- `not` True evaluates to False
- `not` False evaluates to True
- Most useful on a variable that already holds a boolean, rather than on a comparison - `y != 0` already says "not equal" more directly than `not (y == 0)`

```
is_raining = True

if not is_raining:
  print("Let's go for a walk")
else:
  print("Better stay inside")
```

A reference for every value `not`, `and` and `or` can produce:

X	Y	not X	X and Y	X or Y
False	False	True	False	False
False	True	True	False	True
True	False	False	False	True
True	True	False	True	True

For example, X could be "the username is correct" and Y "the password is correct" - `X and Y` is only true when both are:

```
username = input("Username: ")
password = input("Password: ")

if username == "admin" and password == "secret123":
  print("Login successful")
else:
  print("Login failed")
```

Exercise 1

Solve exercise 1

30 min

Conditionals: if/else/elif - Exercise 1

- Write a script that asks for a temperature in °C and prints whether water is solid, liquid or gas
  - below 0 → solid (ice)
  - between 0 and 100 → liquid
  - above 100 → gas (steam)
- Use `input()` to read the temperature
- Print a full sentence, e.g. `At -5 °C water is solid`

Conditionals: if/else/elif - Exercise 1

- `input()` gives you a string, not a number
- Convert it with `float()` before comparing

```
temperature = input('Temperature in degrees Celsius: ')
temperature = float(temperature)
# now compare temperature with 0 and 100
```

- What should your script do at exactly 0 °C or exactly 100 °C?

Exercise 2

Solve exercise 2

15 min

Conditionals: if/else/elif - Exercise 2

- Read three numbers `x`, `y` and `z` with `input()`
- Print `True` if the three numbers are in ascending order, `False` otherwise
- Use `and` to combine the two comparisons
- Then solve it a second time without an `if`: a comparison already is a `True`/`False` value and can be printed directly

Naming Conventions & Debugging

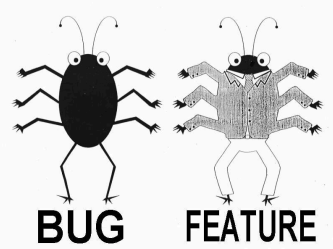
How to name your functions and variables (PEP8)

- Naming convention is a set of rules for choosing names of functions and variables
- Every programming language has different naming conventions
- Python
  - No spaces in variable and function names
  - Variable and function names are in lowercase and _ is used to separate words

```
length_in_cm = 15

def say_hello():
  print("Hello")
```

Finding and resolving “bugs”



- Programming is a complex activity
- Mistakes happen all the time
- A mistake made in programming is called a bug
- The process of finding and resolving bugs is called debugging

Errors

- Syntax error
  - Incorrect syntax of a statement: `print(Hello World)` instead of `print("Hello World")`
- Runtime error
  - Error that occurs during the execution of a program
  - e.g. division by 0

Errors, continued

- Semantic error
  - Program does not deliver correct results
  - No error messages (code is syntactically correct)
  - Fixing semantic errors can be extremely complicated (good software design is important)
  - e.g. averaging two numbers by dividing by 3 instead of 2

```
a = 4
b = 8
average = (a + b) / 3  # should be / 2
print(average)
# 4.0 - runs fine, no error, just wrong
```

Techniques

- Reading code
- Print variables with `print()` to examine values (the simplest debugging tool)
- Go through the program step by step with a **debugger**

Handling errors

- Some errors can be prevented by checking first, like `if key in dict` or `Path.exists()`
- Others cannot - e.g. you cannot know if the user typed a valid number without trying to convert it
- For those, `try`/`except` catches the error instead of crashing

```
while True:
  try:
    age = float(input("Enter your age: "))
    break
  except ValueError:
    print("That's not a number, try again")

print(f"You are {age} years old")
```

Lists: []

Lists: [] - Lists

- Lists are a data type
- Lists are used in most programming languages (vectors)
- Lists are an ordered collection of values

```
exam_scores = [88, 92, 75, 100]
playlist = ["Bohemian Rhapsody", "Imagine", "Hotel California"]
```

Creating lists

- The easiest way to create a list is using []

```
groceries = ["apples", "milk", "bread", "eggs"]
playlist = ["Bohemian Rhapsody", "Imagine", "Hotel California"]
```

- Data types can be mixed

```
cart_item = ["Coffee", 2, 4.50, True]  # name, quantity, price, on sale
```

Creating lists

- Since numbers are often stored in a list, there is a special method for doing so
- With only one argument, range returns a number series starting at 0

```
warmup_laps = list(range(5))
# [0, 1, 2, 3, 4] - the 5 warm-up laps, numbered the way Python counts
```

- When using two arguments it is possible to define the start and end of the range `[start, end)` (end is not included in the list)

```
training_weeks = list(range(1, 5))
# [1, 2, 3, 4] - the weeks before race week, which is week 5
```

Creating lists

- The step size can be defined with a third argument

```
training_days = list(range(1, 10, 2))
# [1, 3, 5, 7, 9] - run every other day, resting in between
```

- An empty list can also be created

```
race_log = []
```

- This is often done when the values to be inserted in the list are not yet known.

Creating lists

- Accessing elements can be done with the [] operator

```
print_queue = ["resume.pdf", "invoice.pdf", "photo.jpg", "essay.docx"]
print(print_queue[1])
# prints invoice.pdf
```

Important

List indices start at 0!

0	1	2	3
resume.pdf	invoice.pdf	photo.jpg	essay.docx

Accessing lists

- A negative index is used to access the list from the end

```
print_queue = ["resume.pdf", "invoice.pdf", "photo.jpg", "essay.docx"]
print(print_queue[-1])
# prints essay.docx - the job that was added most recently
```

Length

- The number of elements in a list can be obtained using the `len()` function

```
print_queue = ["resume.pdf", "invoice.pdf", "photo.jpg", "essay.docx"]
print(len(print_queue))
# prints 4
```

Out of range

- If there is no item in the list at the desired index, Python raises an IndexError

```
print_queue = ["resume.pdf", "invoice.pdf", "photo.jpg", "essay.docx"]
n_jobs = len(print_queue)
print(print_queue[n_jobs])
# IndexError: list index out of range
```

Changing elements in a list

- An element can be changed using [INDEX]

```
print_queue = ["resume.pdf", "invoice.pdf", "photo.jpg", "essay.docx"]
print_queue[0] = "resume_v2.pdf"
# ["resume_v2.pdf", "invoice.pdf", "photo.jpg", "essay.docx"]
```

Adding elements

- The `append()` method can be used to add an element at the end of the list

```
print_queue = ["resume.pdf", "invoice.pdf"]
# ["resume.pdf", "invoice.pdf"]
print_queue.append("photo.jpg")
# ["resume.pdf", "invoice.pdf", "photo.jpg"]
```

Concatenate lists

- The + operator can be used to join lists

```
morning_playlist = ["Here Comes the Sun", "Walking on Sunshine"]
workout_playlist = ["Eye of the Tiger", "Stronger"]
full_playlist = morning_playlist + workout_playlist
# ["Here Comes the Sun", "Walking on Sunshine", "Eye of the Tiger", "Stronger"]
```

Slices

- Lists can be cut into slices
- The operator [n:m] returns a list of the elements that start at index n and stop before m

```
playlist = ["Bohemian Rhapsody", "Imagine", "Hotel California", "Yesterday", "Thriller"]
playlist[1:3]
# ["Imagine", "Hotel California"]
```

Slices

- If the first index is empty, the slice starts at the beginning

```
playlist = ["Bohemian Rhapsody", "Imagine", "Hotel California", "Yesterday", "Thriller"]
playlist[:3]
# ["Bohemian Rhapsody", "Imagine", "Hotel California"] - the top 3 of the set
```

- If the second index is empty, the slice will include elements until the end of the list

```
playlist[3:]
# ["Yesterday", "Thriller"] - the rest of the set
```

Deleting elements

- The `del` statement removes items from the list

```
jobs = ["resume.pdf", "invoice.pdf", "photo.jpg"]
del jobs[1]
# ["resume.pdf", "photo.jpg"] - invoice.pdf was cancelled
queue = ["a.pdf", "b.pdf", "c.pdf", "d.pdf", "e.pdf", "f.pdf"]
del queue[1:5]
# ["a.pdf", "f.pdf"] - printer jammed, whole batch cleared
```

Immutables: Tuples () and Strings

A tuple is an immutable sequence data type

- It is not possible to assign to the individual items of a tuple, however it is possible to create tuples which contain mutable objects, such as lists.
- Tuples are declared using () instead of []

```
my_tuple = ("a", "b", "c", "d", "e")
```

Strings are immutable

- Just like tuples (and unlike lists), strings cannot be changed once created
- Operations on strings always return a modified copy of the string
- The original string remains unchanged

```
greeting = "Hello, world!"
greeting[0] = "J"
# TypeError: 'str' object does not support item assignment
```

Iteration: for/while

Iteration: for/while - Iterations

- Iterations are used to repeat statements
- There are two statements for iterations
  - `while`
  - `for`

`while`

- As long as the condition of the while loop is True, the body of the loop gets executed

Example

```
while n > 0:
  print(n)
  n = n - 1
print('Lift off!')
```

while

- If the condition is False at the beginning, the body of the loop is never executed
- If the variable that is used to check the condition of the while loop does not change, the loop will never terminate -> infinite loop
- Whether a while loop terminates can be hard to determine

```
while n != 1:
  print(n)
  if n % 2 == 0:
    n = n / 2
  else:
    n = n * 3 + 1
```

Iteration: for/while - Iterations

while

- A `while` loop can be used to iterate through a list

```
names = ["Tom", "Anna", "Christopher"]
index = 0
while index < len(names):
  name = names[index]
  print(name)
  index += 1
```

break

- `break` leaves the loop immediately, no matter what the condition says
- Together with `while True` this is the usual way to repeat something until the user wants to stop

```
while True:
  answer = input('Type exit to quit: ')
  if answer == 'exit':
    break
  print(answer)
```

Exercise 3

Solve exercise 3

20 min

Iteration: for/while - Exercise 3

- Keep asking the user for a number with `input()`
- Add each number to a running total
- Stop when the user types `done` instead of a number, then print the total
- Use `while True` together with `break`, just like before

Exercise 4

Solve exercise 4

25 min

Iteration: for/while - Exercise 4

FizzBuzz with a `while` loop

- Create a list with the numbers from 1 to 100: `list(range(1, 101))`
- Write a script that walks through the list in a `while`-loop using an index and prints for each number
  - `FizzBuzz` if it is divisible by 3 and by 5
  - `Fizz` if it is only divisible by 3
  - `Buzz` if it is only divisible by 5
  - the number itself otherwise
- The while loop ends once the end of the list is reached
- Careful: check the FizzBuzz case first, otherwise 15 prints `Fizz`
- Bonus: solve it without ever checking for 15

for

- Since it is often necessary to operate through lists and other data types, there is a special statement for this

```
for element in element_list:
  print(element)
```

Modules & Imports

Modules & Imports

- Besides built-in functions like `print()` and `len()`, many more have already been written for us
- A library (or module) is a collection of such functions, grouped by topic
- `math` is part of Python's standard library and provides common mathematical functions and constants
- Before a library's functions can be used, it has to be imported with `import`

- https://docs.python.org/3/library/math.html

Modules & Imports

`import module`

- Imports the whole module - access its contents with `module.name`

```
import math
result = math.sqrt(16)
```

`from module import name`

- Imports one specific name directly - use it without a prefix

```
from pathlib import Path
file_path = Path("words.txt")
```

`import module as alias`

- Same as `import module`, just under a shorter name

```
import math as m
result = m.sqrt(16)
```

`from module import *`

- Imports everything from the module without a prefix
- Avoid this: it is unclear where a name came from, and it can silently overwrite names you already defined

Reading Data

Reading Data

- So far all the data in our programs was typed directly into the code or entered with `input()`
- Real programs often need to read data that already exists, for example in a text file
- Python can open files on disk and read their content

pathlib

- The `pathlib` module gives a modern, cross-platform way to refer to a file
- `Path.exists()` checks whether the file is actually there

```
from pathlib import Path

file_path = Path('words.txt')
if file_path.exists():
  print(f"File exists: {file_path}")
```

Read: the whole file at once

- `Path.read_text()` reads the entire file as one string
- Use it when the file is needed as a single block, e.g. to pass to `json.loads()`

```
from pathlib import Path

file_path = Path('words.txt')
content = file_path.read_text()
print(content)
```

Read: line by line

- A `for` loop can be used to read a file line by line
- Use it when each line matters on its own, e.g. one word per line in a word list
- `line.strip()` removes the trailing `'\n'`

```
from pathlib import Path

file_path = Path('words.txt')
with file_path.open("r") as file:
  for line in file:
    line = line.strip()
    print(line)
```

Exercise 5

Solve exercise 5

30 min

Reading Data - Exercise 5.1

- Write a script that counts how many words in a list are at least as long as the specified word length and that prints those words
- Use a `for` loop to do this
- Example:

```
words = ['Python', 'Java', 'C', 'Go', 'JavaScript']
min_word_length = 4
counter = 0
# Use a for loop to print every word that is long enough and to count them
# Should print Python, Java, JavaScript and the counter 3
```

Reading Data - Exercise 5.2

- Create a text file called `words.txt` in the same folder as your script, one word per line - use any text editor, not Python
- Extend your script from 5.1 so that it reads the words from `words.txt` instead of the hardcoded list
- Use `Path.exists()` to check the file is there before you try to read it
- The result is the same kind of output as in 5.1, but now for however many words you put in the file

Functions

Functions

- You have already been calling functions: `print`, `input`, `float`, `int`, `len`, `range`, ...
- Now it is time to write your own

User-defined functions

- A function encapsulates some functionality
- Reduces complexity

```
def announce(title, subtitle):
  print(title)
  print(subtitle)
```

- Syntax is important
  - Indentation
  - The colon

Examples

```
def divider():
  print("~~~~~~~~~~~~~~~~~~~~")

print("Level 1: Forest")
divider()
print("Level 2: Cave")
divider()
print("Level 3: Castle")
divider()
print("Level 4: Boss Fight")
```

- If we want a different-looking divider we only need to change a single line of code

```
def divider():
  print("====================")
```

Examples

- For a bigger break between chapters we can define a new function that calls the `divider()` function twice

```
def big_divider():
  divider()
  divider()

print("Chapter 1: The Forest")
big_divider()
print("Chapter 2: The Caves")
```

Parameters and arguments

- Arguments are passed when calling a function
- Value of arguments is assigned to parameters

```
def add_points(current_score, points_earned):
  total = current_score + points_earned
  print(total)

add_points(100, 50)
add_points(0, 25)
```

Parameters and arguments

- Parameters are variables valid within the scope of the function
- Variables that are defined in a function can only be seen inside that function
- Scope can be identified by indentation

```
def combine_score(base, bonus):
  total_score = base + bonus
  print(total_score)

combine_score(50, 20)
print(total_score) # NameError: name 'total_score' is not defined
```

Conclusion

- A function can be called multiple times
- If some code can be reused, put it in a function so you need to write less code
  - Higher factorization
  - Less redundancy
  - Better maintenance
- Functions can also call other functions

Functions - Functions with return value

- We have already used functions that return a value: `input()` gives us back a string, `math.sqrt()` gives us back a number

```
answer = input("Do you like Python?")  # returns a string
root = math.sqrt(16)                   # returns a number
```

- Our own functions have only printed something so far - now it is time to make them return a value too

Functions - Functions with return value

`return`

- Functions that return a value use the `return` keyword

```
import math
def area(radius):
  result = math.pi * radius ** 2
  return result

print(area(10))
my_circle_area = area(8)
```

- Functions can return any valid data type

Boolean return values

- The functions can return a boolean value (True, False)
- The function name should be formulated as a yes/no question
- The return value can be used directly in a condition

```
def is_divisible(x, y):
  return x % y == 0

x = 10
y = 3
if is_divisible(x, y):
    print(f"{x} is divisible by {y}")
else:
    print(f"{x} is not divisible by {y}")
```

Exercise 6

Solve exercise 6

15 min

Functions - Exercise 6

- Copy the `divider()` function from before and add two parameters: `char` and `length`
- It should print `length` copies of `char`, e.g. `divider("~", 20)` prints 20 tildes
- Call it with different characters and lengths to check it works, e.g. `divider("=", 10)` and `divider("-", 5)`
- Hint: Python lets you multiply a string by a number to repeat it - `"ab" * 3` is `"ababab"`

Exercise 7

Solve exercise 7

30 min

Functions - Exercise 7.1

Solve 7.1 to 7.3 in the same file.

- Write a function `total_price(prices)` which takes a **list** of prices as its parameter and returns their sum
- Use a `for` loop and add every price to a running total: `total = total + price`
- `total_price([4.50, 2.00, 13.90])` returns `20.4`

Functions - Exercise 7.2

- Write a function `average_price(prices)` which returns the average price
- Call `total_price()` from inside it - do not write the loop a second time
- `len(prices)` gives you the number of prices

Functions - Exercise 7.3

- Write a function `print_summary(prices)` which prints how many prices there are, the total and the average
- It has to call `total_price()` and `average_price()` instead of calculating anything a second time
- Use `:.2f` so that the amounts are printed with two decimals
- For the list above it prints: `3 items, total 20.40, average 6.80`

Dictionaries: {}

Key-Value pair

- Dictionaries are similar to lists but have a key and value for each entry
- Just like a real dictionary maps a word to its meaning

```
eng2de = {"one": "eins", "two": "zwei", "three": "drei"}
print(eng2de["two"])
# zwei
```

Creating dictionaries

- Dictionaries are created using {}

```
eng2de = {}
eng2de["one"] = "eins"
eng2de["two"] = "zwei"
```

- Values can be added directly

```
inventory = {
"apples": 430,
"bananas": 312,
}
```

Accessing entries

- Values can be accessed directly using `dictionary["key"]`

```
inventory = {
"apples": 430,
"bananas": 312,
}
print(f"Apples in stock: {inventory['apples']}")
# Apples in stock: 430
```

Assigning and modifying values

- The key is assigned a value
- If the key already exists the existing value is overwritten

```
inventory = {
"apples": 430,
"bananas": 312,
}
inventory["oranges"] = 530
inventory["bananas"] = 250
print(f"Bananas in stock: {inventory['bananas']}")
# Bananas in stock: 250
```

Deleting entries

- Key-value pairs can be deleted using the `del` statement

```
inventory = {
"apples": 430,
"bananas": 312,
}
del inventory["bananas"]
print(inventory)
# {'apples': 430}
```

Number of entries

- The `len()` function returns the number of entries

```
inventory = {
  "apples": 430,
  "bananas": 312,
}
print(f"Inventory has {len(inventory)} kinds of fruit")
# Inventory has 2 kinds of fruit
```

Checking if an entry exists

- The `in` keyword can be used to check if a key exists in a dictionary

```
inventory = {
  "apples": 430,
  "bananas": 312,
}
if "apples" in inventory:
  inventory["apples"] += 100
else:
  inventory["apples"] = 100
print(f"Apples in stock: {inventory['apples']}")
# Apples in stock: 530
```

Iterating over entries

- The `items()` method combined with the `for` statement can be used to iterate through every key-value pair

```
inventory = {
  "apples": 430,
  "bananas": 312,
}
for fruit, count in inventory.items():
  print(f"{fruit}: {count}")
# apples: 430
# bananas: 312
```

Exercise 8

Solve exercise 8

60 min

Dictionaries: {} - Exercise 8.1

- Write a function `calculate_mark(points, max_points)` which returns a grade in the Swiss grading scale
- $\text{mark} = \frac{\text{points} \cdot 5}{\text{max points}} + 1$
- `input()` returns strings, so convert both arguments with `float()` inside the function
- The function rounds the grade to the nearest 0.5
  - multiply by 2, round to a whole number and divide by 2 again: `int(mark * 2 + 0.5) / 2`
  - 5.66666 → 5.5
  - 5.75 → 6

Dictionaries: {} - Exercise 8.2

- Ask for max_points via the `input()` function
- Write code that in a `while` loop asks for points as long as the user does not enter "exit"
- `break` leaves the loop immediately
- The grade should be printed after each run

```
# Read max_points from input
while True:
    # Read points from input
    if points == 'exit':
        break
    # call calculate_mark function
    # print result
```

Dictionaries: {} - Exercise 8.3

- Change your code so that it additionally asks for a name
- A dictionary should now store the grade of each name
  - The name is a key, the grade the value
- As soon as the user enters "exit" the program should print the grades of all names before it exits

Dictionaries: {} - Exercise 8.4

- Change your code in such a way that for each name it additionally outputs if the user has passed or failed
- Mark >= 4 $\longrightarrow$ passed
- Mark < 4 $\longrightarrow$ failed
- Write a function `has_passed(mark)` which returns True or False and use it in the `if`

Dictionaries: {} - Exercise 8.5

- Change your code in such a way that the application outputs the average grade before it exits
- Print the average with two decimals (`:.2f`)
- Make sure the application does not crash when the user exits without entering a single grade
- Bonus: also print the name and grade of the best and the worst student

Exercise 9

Solve exercise 9

35 min

Dictionaries: {} - Exercise 9

- Write an application that generates a random number between 1 and 100
- `import random`
- `random.randrange(1, 101)`: the upper bound is not part of the range
- The user makes a guess and enters a number. If the number is incorrect, the program outputs whether the entered number was too small or too large and allows the user to guess again.
- The application quits when the correct number is guessed
- The application should output how many user attempts have been made before it quits
- Bonus: ask the user after every round whether they want to play again

Exercise 10

Solve exercise 10

Stretch goal - if you finish exercise 9 early

Dictionaries: {} - Exercise 10

- Implement the opposite of exercise 9: the user enters a number and the computer has to find it
- The computer starts with a random guess between 1 and 100
- Two variables `lower` and `upper` hold the range the number can still be in
- Narrow that range after every guess and guess in the middle of what is left
- Print every guess and how many guesses were needed in the end
- How many guesses does the computer need in the worst case? Why?

Dictionaries: {} - Exercise 10

A possible skeleton:

```
lower = 1
upper = 100
guess = random.randrange(1, 101)
while guess != target_number:
    # print the guess
    # move lower or upper to just past the guess
    guess = (lower + upper) // 2  # // divides and drops the remainder
```

4

Persistence

Persistence

- You already know how to read data - now it's time to write it
- So far, none of our own programs has saved anything: all data was lost as soon as the program quit
- There are several ways to store data permanently on disk
  - Database
  - Simple text files

Persistence - Writing Data

Common procedure

- Open file
- Do something with the file
- Close the file - depending on syntax done automatically.

```
from pathlib import Path

file_path = Path('my_file.txt')
with file_path.open("mode") as file:
  ...  # do something with the file
```

Different modes

- The mode defines how the content of the file should be treated
- Modes
  - `"r"`: read only
  - `"w"`: write only
  - `"r+"`: read and write
  - `"a"`: append

```
from pathlib import Path

file_path = Path('my_file.txt')
# open a file in write mode
with file_path.open("w") as file:
  ...  # do something with the file
```

Persistence - Writing Data

Write

- The write_text() method is a convenient way to write strings
- The open() method with `"a"` mode can be used for appending
- `'\n'` is used to insert a line break

```
from pathlib import Path

file_path = Path('my_file.txt')

# Simple write (overwrites file)
file_path.write_text('Hello World\n')

# Append mode
with file_path.open("a") as file:
  file.write('First line of the write operation\n')
  file.write('This is a line with a new-line character at the end\n')
```

Dictionaries/list in JSON

- file.write() only accepts strings as arguments
- If complex structures such as dictionaries or lists should be stored in a file, it's necessary to convert these structures into strings first
- An example of a standard used for this purpose is JSON (JavaScript Object Notation)

```
import json
my_dict = {"one": "eins", "two": "zwei"}
my_dict_as_string = json.dumps(my_dict)
print(my_dict_as_string)
```

Convert JSON to dictionaries/lists

- Example of a string in JSON that is converted into a dictionary

```
import json
my_dict_as_string = '{"two": "zwei", "one": "eins"}'
my_dict = json.loads(my_dict_as_string)
print(my_dict)
```

Combining pathlib with JSON

- Path makes it easy to save and load JSON files
- write_text() and read_text() work perfectly with JSON

```python
import json
from pathlib import Path

# Save dictionary to JSON file
my_dict = {"one": "eins", "two": "zwei", "three": "drei"}
file_path = Path('data.json')
file_path.write_text(json.dumps(my_dict, indent=2))

# Load dictionary from JSON file
loaded_dict = json.loads(file_path.read_text())
print(loaded_dict)
```

Exercise 11

Solve exercise 11

40 min

Persistence - Exercise 11

- Write an application which repeatedly asks for a name and phone number until the user enters “exit”
- Each name/telephone number pair should be stored as an entry in a dictionary
  - The names are the keys of the dictionary
  - The telephone numbers are the values of the dictionary
- As soon as the user enters “exit”, write the dictionary to a file called address_book.txt with `json.dumps()` and `Path.write_text()`

Persistence - Exercise 11

- Extend your application so that it reads the address_book.txt file when it starts and converts the JSON text into a dictionary again
- On the very first run the file does not exist yet, so check with `path.exists()` before reading

```
import json
from pathlib import Path

path = Path('address_book.txt')
address_book = {}
if path.exists():
    address_book = json.loads(path.read_text())
```

- Let the user choose: add a person, search for a phone number, remove a person, or exit
- Print a hint when the input is none of these
- Write the dictionary back to address_book.txt before the application quits

Additional Resources

- Think Python, 3rd edition, by Allen Downey - free, interactive (runs in Colab), and covers Python 3
  - https://allendowney.github.io/ThinkPython/
- Official Python Documentation
  - https://docs.python.org/3/
- Project Euler: Mathematical problems that can be solved programmatically
  - https://projecteuler.net/
- CodinGame: Learn and practice coding by solving puzzles and playing games
  - https://www.codingame.com/start/
- Platforms to prepare for coding interviews
  - https://leetcode.com
  - https://www.hackerrank.com/

VS Code Shortcuts

`Cmd` on Mac, `Ctrl` on Windows/Linux:

Shortcut	Action
`Cmd/Ctrl + S`	Save the file
`Cmd/Ctrl + ` `	Open/close the integrated terminal
`Cmd/Ctrl + Shift + P`	Open the command palette
`Cmd/Ctrl + P`	Quickly open a file by name
`Cmd/Ctrl + /`	Comment/uncomment the current line
`Cmd/Ctrl + F`	Find text in the current file
`Cmd/Ctrl + D`	Select the next occurrence of the current word
`Cmd/Ctrl + Z`	Undo (add Shift to redo)

Setting Up Ruff

Ruff

- A very fast Python linter and code formatter, usable directly inside VS Code
- Install the **Ruff** extension from the VS Code Extensions panel (publisher: Astral Software)
- Then open `Preferences: Open User Settings (JSON)` from the command palette and add:

```
{
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
    "editor.formatOnSave": true
  }
}
```

- From now on, every time you save a Python file it gets automatically formatted

© 2026 Bytes & Beyond GmbH. All rights reserved.